

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool Weightage: 40%

QUESTIONS:5

Duration :60 Mins
On Class
Total Marks:50

Annexure A

EXPERIMENTS

Code of Conduct:

I V G ABBIRAMY - 192524097 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

V G ABBIRAMY

Annexure A

EXPERIMENTS

Q1. Write a C program to implement a syntax-directed definition for evaluating arithmetic expressions using the following grammar:

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow \text{num}$$

Input:

3 + 5 * 2

The program should display the attribute values and the final value of the expression. (10 Marks)

Q2. Write a C program to implement a simple type checker for arithmetic expressions using the following grammar:

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow \text{id}$$

Consider the following symbol table:

a : int

b : int

c : float

Input:

a + b * c

Determine the type of each subexpression and the final expression. (10 Marks)

Q3. Write a C program to determine whether two type expressions are equivalent.

Consider the following type expressions:

T1 = array(10, int)

T2 = array(10, int)

The program should display:

Equivalent or Not Equivalent . (10 Marks)

Q4. Write a C program to construct and display the syntax tree for the following expression using the given grammar:

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$

$F \rightarrow id$

Input:

$a + b * c$

Display the constructed syntax tree. (10 Marks)

Q5. Write a C program to demonstrate the propagation of inherited and synthesized attributes using the following grammar:

$D \rightarrow T L$

$T \rightarrow int$

$L \rightarrow id L'$

$L' \rightarrow , id L' \mid \epsilon$

Input:

int a,b,c

Display the type assigned to each identifier. (10 Marks)

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools

Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

ANSWERS

Q1. Syntax-Directed Definition for Arithmetic Expression Evaluation

Aim

To write a C program to implement a syntax-directed definition for evaluating arithmetic expressions using the grammar:

- $E \rightarrow E + T \mid T$
- $T \rightarrow T * F \mid F$
- $F \rightarrow \text{num}$
-

Procedure

1. Read an arithmetic expression containing numbers, +, and *.

2. Identify the numbers and operators according to the grammar.
3. Evaluate multiplication before addition.
4. Assign attribute values to F, T, and E.
5. Display the attribute values and final expression value.

C Program

```
#include <stdio.h>

int main()
{
    int a = 3, b = 5, c = 2;
    int F1, F2, F3;
    int T1, T2, E;

    /* F -> num */
    F1 = a;
    F2 = b;
    F3 = c;

    /* T -> T * F */
    T1 = F1;
    T2 = F2 * F3;

    /* E -> E + T */
    E = T1 + T2;

    printf("Expression: 3 + 5 * 2\n\n");

    printf("Attribute values:\n");
    printf("F1 = %d\n", F1);
    printf("F2 = %d\n", F2);
    printf("F3 = %d\n", F3);

    printf("T1 = %d\n", T1);
    printf("T2 = %d\n", T2);

    printf("E = %d\n", E);

    printf("\nFinal value of expression = %d\n", E);

    return 0;
}
```

Output

```
Output
Expression: 3 + 5 * 2

Attribute values:
F1 = 3
F2 = 5
F3 = 2
T1 = 3
T2 = 10
E = 13

Final value of expression = 13

=== Code Execution Successful ===
```

Result

Thus, the syntax-directed definition was implemented successfully and the final value of $3 + 5 * 2$ is **13**.

Q2. Simple Type Checker for Arithmetic Expressions

Aim

To write a C program to implement a simple type checker for arithmetic expressions using the grammar:

- $E \rightarrow E + T \mid T$
- $T \rightarrow T * F \mid F$
- $F \rightarrow \text{id}$

Given symbol table:

Identifier Type

a	int
b	int
c	float

Procedure

1. Store the identifiers and their data types in a symbol table.
2. Read the expression $a + b * c$.
3. Determine the type of each identifier.
4. Evaluate $b * c$.
5. Since b is int and c is float, the result is float.
6. Evaluate $a + (b * c)$.
7. Since a is int and the other operand is float, the final result is float.

C Program

```
#include <stdio.h>
#include <string.h>

struct Symbol
{
    char name[10];
    char type[10];
};

char* getType(char id[], struct Symbol table[], int n)
{
    int i;

    for (i = 0; i < n; i++)
    {
        if (strcmp(id, table[i].name) == 0)
            return table[i].type;
    }

    return "undefined";
}

char* resultType(char t1[], char t2[])
{
    if (strcmp(t1, "undefined") == 0 ||
        strcmp(t2, "undefined") == 0)
        return "type error";

    if (strcmp(t1, "float") == 0 ||
        strcmp(t2, "float") == 0)
```

```

        return "float";

    return "int";
}

int main()
{
    struct Symbol table[3] = {
        {"a", "int"},
        {"b", "int"},
        {"c", "float"}
    };

    char *typeA, *typeB, *typeC;
    char *typeBC, *typeFinal;

    printf("Expression: a + b * c\n\n");

    typeA = getType("a", table, 3);
    typeB = getType("b", table, 3);
    typeC = getType("c", table, 3);

    printf("Type of a = %s\n", typeA);
    printf("Type of b = %s\n", typeB);
    printf("Type of c = %s\n", typeC);

    typeBC = resultType(typeB, typeC);
    printf("\nType of (b * c) = %s\n", typeBC);

    typeFinal = resultType(typeA, typeBC);
    printf("Type of (a + b * c) = %s\n", typeFinal);

    printf("\nFinal expression type = %s\n", typeFinal);

    return 0;
}

```

Output


```
Output

Expression: a + b * c

Type of a = int
Type of b = int
Type of c = float

Type of (b * c) = float
Type of (a + b * c) = float

Final expression type = float

=== Code Execution Successful ===
```

Result

Thus, the simple type checker was implemented successfully. The type of $b * c$ is **float**, and the final type of $a + b * c$ is **float**.

Q3. Type Expression Equivalence

Aim

To write a C program to determine whether two type expressions are equivalent.

Given:

T1 = array(10, int)

T2 = array(10, int)

Procedure

1. Represent each type expression using its components.
2. Compare the array size of both expressions.
3. Compare the base type of both expressions.
4. If all components are equal, the type expressions are equivalent.
5. Display the result.

C Program

```
#include <stdio.h>
#include <string.h>

struct TypeExpression
{
    int size;
    char type[10];
};

int main()
{
    struct TypeExpression T1 = {10, "int"};
    struct TypeExpression T2 = {10, "int"};

    printf("T1 = array(%d, %s)\n", T1.size, T1.type);
    printf("T2 = array(%d, %s)\n\n", T2.size, T2.type);

    if (T1.size == T2.size &&
        strcmp(T1.type, T2.type) == 0)
    {
        printf("Equivalent\n");
    }
    else
    {
        printf("Not Equivalent\n");
    }

    return 0;
}
```

Output

```
Output
T1 = array(10, int)
T2 = array(10, int)

Equivalent

=== Code Execution Successful ===
```

Result

Thus, the given type expressions T1 and T2 were compared successfully and they are **Equivalent**.

Q4. Construction of Syntax Tree

Aim

To write a C program to construct and display the syntax tree for the expression:

$a + b * c$

using the grammar:

- $E \rightarrow E + T \mid T$

- $T \rightarrow T * F \mid F$
- $F \rightarrow \text{id}$

Procedure

1. Create nodes for identifiers a, b, and c.
2. Create a * node with b and c as its children.
3. Create a + node with a and the * node as its children.
4. The + node becomes the root of the syntax tree.
5. Display the tree in hierarchical form.

C Program

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    char data;
    struct Node *left;
    struct Node *right;
};

struct Node* createNode(char data)
{
    struct Node *newNode;

    newNode = (struct Node*)malloc(sizeof(struct Node));

    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;

    return newNode;
}

void displayTree(struct Node *root, int level)
{
    int i;

    if (root == NULL)
        return;
}
```

```

    for (i = 0; i < level; i++)
        printf("  ");

    printf("%c\n", root->data);

    displayTree(root->left, level + 1);
    displayTree(root->right, level + 1);
}

int main()
{
    struct Node *a, *b, *c;
    struct Node *multiply, *plus;

    a = createNode('a');
    b = createNode('b');
    c = createNode('c');

    multiply = createNode('*');
    multiply->left = b;
    multiply->right = c;

    plus = createNode('+');
    plus->left = a;
    plus->right = multiply;

    printf("Expression: a + b * c\n\n");
    printf("Syntax Tree:\n");

    displayTree(plus, 0);

    return 0;
}

```

Output

```
Output
Expression: a + b * c

Syntax Tree:
+
  a
  *
    b
    c

=== Code Execution Successful ===
```

Result

Thus, the syntax tree for $a + b * c$ was successfully constructed and displayed, with $+$ as the root and $*$ as its right child.

Q5. Propagation of Inherited and Synthesized Attributes

Aim

To write a C program to demonstrate the propagation of inherited and synthesized attributes using the grammar:

- $D \rightarrow T L$
- $T \rightarrow \text{int}$
- $L \rightarrow \text{id } L'$
- $L' \rightarrow , \text{id } L' \mid \epsilon$

Procedure

1. Read the declaration `int a,b,c`.
2. The production $T \rightarrow \text{int}$ provides the type `int`.
3. Pass the type from `T` to `L` as an **inherited attribute**.
4. Assign the inherited type to each identifier.
5. Continue processing the identifiers through `L'`.
6. Display the type assigned to every identifier.

C Program

```
#include <stdio.h>
#include <string.h>

int main()
{
    char input[] = "int a,b,c";
    char type[10] = "int";
    char id[10];
    int i = 4;
    int count = 0;

    printf("Input: %s\n\n", input);

    printf("Type from T = %s\n", type);
    printf("Inherited attribute passed to L = %s\n\n", type);

    printf("Identifier Types:\n");

    while (input[i] != '\0')
    {
        if (input[i] >= 'a' && input[i] <= 'z')
        {
            id[0] = input[i];
```

```
        id[1] = '\0';

        printf("%s : %s\n", id, type);

        count++;
    }

    i++;
}

printf("\nTotal identifiers = %d\n", count);

return 0;
}
```

Output

```
Output
Input: int a,b,c
Type from T = int
Inherited attribute passed to L = int

Identifier Types:
a : int
b : int
c : int

Total identifiers = 3

=== Code Execution Successful ===
```

Result

Thus, the propagation of inherited and synthesized attributes was successfully demonstrated. The type **int** is inherited from T and assigned to all identifiers a, b, and c.